

WHY SLIDING WINDOW?

Computing subarray sums or frequencies naively takes $O(N^2)$ or $O(N^3)$ time. Sliding Window re-uses computation from overlapping adjacent windows to achieve $O(N)$ time.

THE AHA MOMENT

When moving from window `[left..right]` to `[left+1..right+1]`, do NOT recompute from scratch! Subtract `nums[left]` and add `nums[right+1]` in $O(1)$!

3 CORE VARIANTS

- Fixed Window Size K:** Window size K is fixed. Expand `right`, contracts `left` when `right - left + 1 > K`.
- Dynamic Window Longest:** Expand `right` to include elements. Shrink `left` ONLY when constraint is violated.
- Dynamic Window Shortest:** Expand `right` until constraint is satisfied. Shrink `left` greedily to minimize size.

2 VISUALIZING WINDOW SLIDING

Fixed Window K=3:

Window 1: [2, 1, 5], 1, 3, 2
Sum = 8
 Subtract 2, Add 1 →
Window 2: 2, [1, 5, 1], 3, 2
Sum = 7
 Subtract 1, Add 3 →
Window 3: 2, 1, [5, 1, 3], 2
Sum = 9 (Max!)

PREREQUISITES & COMPLEXITY REASONING

Variant	Time Complexity	Auxiliary Space
Fixed Size K	$O(N)$	$O(1)$
Dynamic Window (Longest)	$O(N)$	$O(1)$ or $O(K)$ freq array
Dynamic Window (Shortest)	$O(N)$	$O(1)$ or $O(K)$ map
At Most K Transformation	$O(N)$	$O(K)$

CLUES THAT SCREAM SLIDING WINDOW

- "Contiguous subarray or substring" + "Maximum / Minimum / Target sum"
- "Longest substring with at most K distinct characters"
- "Find all anagrams in string" → Fixed Window Frequency Match
- "Subarrays with exactly K distinct" → `atMost(K) - atMost(K - 1)`

1 FAST FREQUENCY ARRAY (`int[128]` vs `int[26]`)

```
// Fast ASCII Frequency Tracker (Replaces HashMap for O(1) performance!)
int[] freq = new int[128]; // Fits all ASCII characters

// Add character at right pointer
freq[s.charAt(right)]++;

// Remove character at left pointer
freq[s.charAt(left)]--;
```

💡 Avoids HashMap auto-boxing overhead. Executes 5x faster in FAANG benchmarks!

3 MATCH COUNT TRACKER (Minimum Window Substring)

```
int[] count = new int[128];
int matched = 0; // Number of unique chars meeting frequency target
for (char c : target.toCharArray()) count[c]++;

// During right expansion:
if (--count[s.charAt(right)] == 0) matched++;
```

2 AT MOST K TO EXACTLY K TRICK

```
// Mathematical Identity for "Exactly K" Subarrays:
// exactly(K) = atMost(K) - atMost(K - 1)

public int numSubarraysWithK(int[] nums, int k) {
    return atMost(nums, k) - atMost(nums, k - 1);
}
```

Converts complex "Exactly K" problem into two simple "At Most K" sliding window passes!

4 FREQUENCY ARRAY MATCH CHECK (Anagrams)

```
public static boolean matches(int[] f1, int[] f2) {
    for (int i = 0; i < 26; i++) {
        if (f1[i] != f2[i]) return false;
    }
    return true;
}
```

$O(26) = O(1)$ time comparison between fixed size window and target string!

PATTERN1	FIXED WINDOW SIZE K (Add Right, Remove Left) e.g. Maximum Average Subarray I (LC 643), Maximum Sum Subarray Size K
WHEN TO USE Subarray length \$K\$ is fixed in problem description. Build first window of size \$K\$, then slide across array. 💡 AHA Maintain `sum += nums[i]`. Once `i >= K - 1`, update result, then subtract `nums[i - K + 1]`.	TEMPLATE — MAXIMUM AVERAGE SUBARRAY (LC 643) <pre>public double findMaxAverage(int[] nums, int k) { double sum = 0; for (int i = 0; i < k; i++) sum += nums[i]; double max = sum; for (int i = k; i < nums.length; i++) { sum += nums[i] - nums[i - k]; max = Math.max(max, sum); } return max / k; }</pre>
PATTERN2	FIXED WINDOW FREQUENCY MATCHING (Anagrams & Permutations) e.g. Permutation in String (LC 567), Find All Anagrams (LC 438)
WHEN TO USE Finding subsegment of \$S2\$ that is an anagram/permutation of \$S1\$. Window length is fixed at `s1.length()`.	TEMPLATE — FIND ALL ANAGRAMS (LC 438) <pre>public List<Integer> findAnagrams(String s, String p) { List<Integer> res = new ArrayList<>(); if (s.length() < p.length()) return res; int[] pCount = new int[26], sCount = new int[26]; for (int i = 0; i < p.length(); i++) { pCount[p.charAt(i) - 'a']++; sCount[s.charAt(i) - 'a']++; } if (Arrays.equals(pCount, sCount)) res.add(0); for (int i = p.length(); i < s.length(); i++) { sCount[s.charAt(i) - 'a']++; sCount[s.charAt(i - p.length()) - 'a']--; if (Arrays.equals(pCount, sCount)) res.add(i - p.length() + 1); } return res; }</pre>

PATTERN3 DYNAMIC WINDOW — LONGEST SUBSTRING (Expand Right, Shrink Left) WHEN TO USE Find maximum length subarray/substring satisfying a condition (e.g. all unique characters, max \$K\$ replacements).	TEMPLATE — LC 3 <pre>public int lengthOfLongestSubstring(String s) { int[] map = new int[128]; int left = 0, maxLen = 0; for (int right = 0; right < s.length(); right++) { char c = s.charAt(right); map[c]++; while (map[c] > 1) { // Constraint violated -> shrink left map[s.charAt(left)]--; left++; } maxLen = Math.max(maxLen, right - left + 1); } return maxLen; }</pre> e.g. Longest Substring Without Repeating Characters (LC 3)
PATTERN4 LONGEST SUBSTRING WITH CHARACTER REPLACEMENT OPTIMIZATION INTUITION Valid window if `(windowLength - maxFreq) <= K`. We track `maxCount` of any character seen so far. If window becomes invalid, shrink left by 1!	TEMPLATE — LC 424 <pre>public int characterReplacement(String s, int k) { int[] count = new int[26]; int left = 0, maxCount = 0, maxLen = 0; for (int right = 0; right < s.length(); right++) { maxCount = Math.max(maxCount, ++count[s.charAt(right) - 'A']); if (right - left + 1 - maxCount > k) { count[s.charAt(left) - 'A']--; left++; } maxLen = Math.max(maxLen, right - left + 1); } return maxLen; }</pre> e.g. Longest Repeating Character Replacement (LC 424)

PATTERN
5

DYNAMIC WINDOW — SHORTEST SUBSTRING (Minimum Window Substring)

e.g. Minimum Window Substring (LC 76)

GREEDY SHRINK MECHANICS

1. Expand `right` until window contains all required chars.
2. Once valid, record min length and shrink `left` as much as possible while maintaining validity!

TEMPLATE — LC 76

```
public String minWindow(String s, String t) {  
    int[] count = new int[128];  
    for (char c : t.toCharArray()) count[c]++;  
    int left = 0, minLen = Integer.MAX_VALUE, start = 0, matched = t.length();  
    for (int right = 0; right < s.length(); right++) {  
        if (count[s.charAt(right)]-- > 0) matched--;  
        while (matched == 0) {  
            if (right - left + 1 < minLen) { minLen = right - left + 1; start = left; }  
            if (++count[s.charAt(left++)] > 0) matched++;  
        }  
    }  
    return minLen == Integer.MAX_VALUE ? "" : s.substring(start, start + minLen);  
}
```

PATTERN
6

AT MOST K DISTINCT SUBARRAYS (Counting Subarrays)

e.g. Subarrays with K Distinct Integers (LC 992), Fruit Into Baskets (LC 904)

COUNT SUBARRAYS FORMULA

Number of valid ending subarrays at `right` is `right - left + 1`.
`exactly(K) = atMost(K) - atMost(K - 1)`.

TEMPLATE — SUBARRAYS WITH K DISTINCT

```
public int subarraysWithKDistinct(int[] nums, int k) {  
    return atMost(nums, k) - atMost(nums, k - 1);  
}  
private int atMost(int[] nums, int k) {  
    Map<Integer, Integer> count = new HashMap<>();  
    int left = 0, res = 0;  
    for (int right = 0; right < nums.length; right++) {  
        if (count.containsKey(nums[right]) == 0) k--;  
        count.put(nums[right], count.getOrDefault(nums[right], 0) + 1);  
        while (k < 0) {  
            count.put(nums[left], count.get(nums[left]) - 1);  
            if (count.get(nums[left]) == 0) k++;  
            left++;  
        }  
        res += right - left + 1;  
    }  
    return res;  
}
```

PATTERN 7

MONOTONIC DEQUE SLIDING WINDOW (Sliding Window Maximum) e.g. Sliding Window Maximum (LC 239)

MONOTONIC DEQUE INTUITION

Maintain a Deque storing indices in **decreasing order of values**.

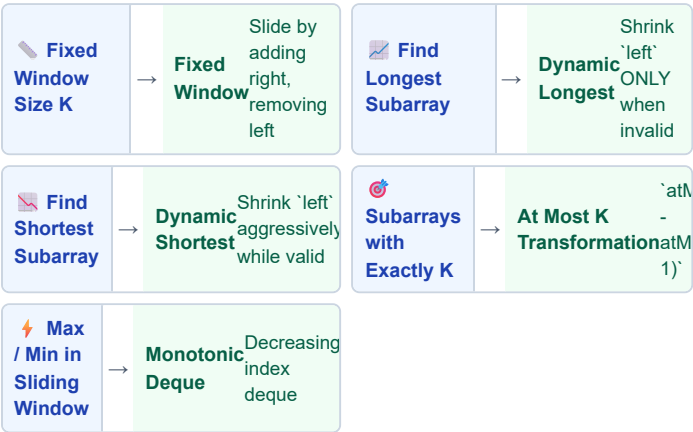
- 1. Remove elements from tail smaller than `nums[right]`.
- 2. Remove head if it fell outside current window (`q.peek() < right - K + 1`).
- 3. Head `q.peek()` is ALWAYS max element of window!

TEMPLATE — LC 239

```
public int[] maxSlidingWindow(int[] nums, int k) {
    if (nums == null || k <= 0) return new int[0];
    int n = nums.length;
    int[] res = new int[n - k + 1];
    Deque<Integer> q = new ArrayDeque<>(); // stores indices
    for (int i = 0; i < n; i++) {
        // 1. Remove indices out of window bounds
        while (!q.isEmpty() && q.peek() < i - k + 1) q.poll();
        // 2. Remove smaller values from tail
        while (!q.isEmpty() && nums[q.peekLast()] < nums[i]) q.pollLast();
        // 3. Offer current index
        q.offer(i);
        // 4. Record result once window reaches size K
        if (i >= k - 1) res[i - k + 1] = nums[q.peek()];
    }
    return res;
}
```

SLIDING WINDOW — WHICH VARIANT TO USE?

Start: What is the window constraint?



TRIGGER WORDS DIRECTORY

Trigger Word	Variant	Action
"Subarray size K"	Fixed Window	<code>`nums[i] - nums[i-k]`</code>
"Longest substring without repeating"	Dynamic Longest	Shrink when <code>`count > 1`</code>
"Minimum window substring"	Dynamic Shortest	Shrink while <code>`matched == target`</code>
"Exactly K distinct elements"	At Most K Difference	<code>`atMost(K) - atMost(K-1)`</code>

INTERVIEW TRADE-OFFS & FOLLOW-UPS

Interviewer Asks	Your Answer
Why use <code>`int[128]`</code> over <code>`HashMap`</code> ?	Array index access is $O(1)$ without hashing or object allocation overhead!
Why <code>`atMost(K) - atMost(K-1)`</code> ?	Counting "Exactly K" directly requires tracking complex left pointer states. Difference of two "At Most" runs is clean $O(N)$!

EASY & MEDIUM — Fixed & Unique Substrings

LC 643 · Max Average Subarray

EASY

Pattern: Fixed Window K
Key Insight: Sum first K, then add right, sub left.

```
public double findMaxAverage(int[] nums, int k) {
    double sum = 0;
    for (int i = 0; i < k; i++) sum += nums[i];
    double max = sum;
    for (int i = k; i < nums.length; i++) {
        sum += nums[i] - nums[i - k];
        max = Math.max(max, sum);
    }
    return max / k;
}
```

LC 3 · Longest Substring Without Repeating

MEDIUM

Pattern: Dynamic Longest
Key Insight: Shrink left when `freq[c] > 1`.

```
public int lengthOfLongestSubstring(String s) {
    int[] map = new int[128];
    int left = 0, maxLen = 0;
    for (int right = 0; right < s.length(); right++) {
        char c = s.charAt(right); map[c]++;
        while (map[c] > 1) map[s.charAt(left++)]--;
        maxLen = Math.max(maxLen, right - left + 1);
    }
    return maxLen;
}
```

LC 424 · Character Replacement

MEDIUM

Pattern: Dynamic Longest
Key Insight: `(len - maxCount) > K`.

```
public int characterReplacement(String s, int k) {
    int[] count = new int[26];
    int l = 0, maxC = 0, maxLen = 0;
    for (int r = 0; r < s.length(); r++) {
        maxC = Math.max(maxC, ++count[s.charAt(r) - 'A']);
        if (r - l + 1 - maxC > k) count[s.charAt(l++) - 'A']--;
        maxLen = Math.max(maxLen, r - l + 1);
    }
    return maxLen;
}
```

MEDIUM — Frequency Anagrams & Fruit Baskets

LC 567 · Permutation in String

MEDIUM

Pattern: Fixed Frequency Match
Key Insight: Fixed window size = `s1.length()`.

```
public boolean checkInclusion(String s1, String s2) {
    if (s1.length() > s2.length()) return false;
    int[] c1 = new int[26], c2 = new int[26];
    for (int i = 0; i < s1.length(); i++) {
        c1[s1.charAt(i) - 'a']++; c2[s2.charAt(i) - 'a']++;
    }
    if (Arrays.equals(c1, c2)) return true;
    for (int i = s1.length(); i < s2.length(); i++) {
        c2[s2.charAt(i) - 'a']++; c2[s2.charAt(i - s1.length()) - 'a']--;
        if (Arrays.equals(c1, c2)) return true;
    }
    return false;
}
```

LC 904 · Fruit Into Baskets

MEDIUM

Pattern: Dynamic Window (At Most 2)
Key Insight: Max length with at most 2 distinct elements.

```
public int totalFruit(int[] fruits) {
    Map<Integer, Integer> map = new HashMap<>();
    int l = 0, max = 0;
    for (int r = 0; r < fruits.length; r++) {
        map.put(fruits[r], map.containsKey(fruits[r]) ? map.get(fruits[r]) + 1 : 1);
        while (map.size() > 2) {
            map.put(fruits[l], map.get(fruits[l]) - 1);
            if (map.get(fruits[l]) == 0) map.remove(fruits[l]);
            l++;
        }
        max = Math.max(max, r - l + 1);
    }
    return max;
}
```

MEDIUM & HARD — Minimum Window & Exactly K

LC 438 · Find All Anagrams

MEDIUM

Pattern: Fixed Frequency Window
Key Insight: Compare ``c1`` & ``c2`` arrays.

```
public List<Integer> findAnagrams(String s, String p) {
    List<Integer> res = new ArrayList<>();
    if (s.length() < p.length()) return res;
    int[] c1 = new int[26], c2 = new int[26];
    for (int i = 0; i < p.length(); i++) {
        c1[p.charAt(i) - 'a']++; c2[s.charAt(i) - 'a']++;
    }
    if (Arrays.equals(c1, c2)) res.add(0);
    for (int i = p.length(); i < s.length(); i++) {
        c2[s.charAt(i) - 'a']++; c2[s.charAt(i - p.length()) - 'a']--;
        if (Arrays.equals(c1, c2)) res.add(i - p.length() + 1);
    }
    return res;
}
```

LC 76 · Minimum Window Substring

HARD

Pattern: Dynamic Shortest
Key Insight: Shrink left while ``matched == t.length()``.

```
public String minWindow(String s, String t) {
    int[] cnt = new int[128]; for (char c : t.toCharArray()) cnt[c]++;
    int l = 0, minL = Integer.MAX_VALUE, start = 0, m = t.length();
    for (int r = 0; r < s.length(); r++) {
        if (cnt[s.charAt(r)]-- > 0) m--;
        while (m == 0) {
            if (r - l + 1 < minL) { minL = r - l + 1; start = l; }
            if (++cnt[s.charAt(l++)] > 0) m++;
        }
    }
    return minL == Integer.MAX_VALUE ? "" : s.substring(start, start + minL);
}
```

HARD — Monotonic Deque & Subarray Count

LC 992 · Subarrays with K Distinct

HARD

Pattern: At Most K Difference
Key Insight: ``atMost(K) - atMost(K-1)``.

```
public int subarraysWithKDistinct(int[] nums, int k) {
    return atMost(nums, k) - atMost(nums, k - 1);
}

private int atMost(int[] nums, int k) {
    Map<Integer, Integer> cnt = new HashMap<>();
    int l = 0, res = 0;
    for (int r = 0; r < nums.length; r++) {
        if (cnt.containsKey(nums[r]) == 0) k--;
        cnt.put(nums[r], cnt.getOrDefault(nums[r], 0) + 1);
        while (k < 0) {
            cnt.put(nums[l], cnt.get(nums[l]) - 1);
            if (cnt.get(nums[l]) == 0) k++; l++;
        }
        res += r - l + 1;
    }
    return res;
}
```

LC 239 · Sliding Window Maximum

HARD

Pattern: Monotonic Deque
Key Insight: Decreasing deque of indices.

```
public int[] maxSlidingWindow(int[] nums, int k) {
    int n = nums.length; int[] res = new int[n - k + 1];
    Deque<Integer> q = new ArrayDeque<>();
    for (int i = 0; i < n; i++) {
        while (!q.isEmpty() && q.peek() < i - k + 1) q.poll();
        while (!q.isEmpty() && nums[q.peekLast()] < nums[i]) q.pollLast();
        q.offer(i);
        if (i >= k - 1) res[i - k + 1] = nums[q.peek()];
    }
    return res;
}
```

COMPLETE SLIDING WINDOW PROBLEM LADDER SUMMARY					
LC #	Problem	Pattern	Time	Space	Diff
643	Max Average Subarray	Fixed Window K	O(N)	O(1)	E
3	Longest Substring Without Repeating	Dynamic Longest	O(N)	O(1)	M
424	Character Replacement	Dynamic Longest	O(N)	O(1)	M
567	Permutation in String	Fixed Frequency Match	O(N)	O(1)	M
904	Fruit Into Baskets	Dynamic Window (At Most 2)	O(N)	O(1)	M
438	Find All Anagrams	Fixed Frequency Match	O(N)	O(1)	M
76	Minimum Window Substring	Dynamic Shortest	O(N)	O(1)	H

🔍 DRY RUN — LC 3 Longest Substring ("abcabcb")

Right	Char	Map State	Left	Window Len	Max Len
0	'a'	a:1	0	"a" (len 1)	1
1	'b'	a:1, b:1	0	"ab" (len 2)	2
2	'c'	a:1, b:1, c:1	0	"abc" (len 3)	3
3	'a'	a:2, b:1, c:1 $\rightarrow$ shrink left!	1	"bca" (len 3)	3

✅ MASTERY CHECKLIST — CAN YOU DO THIS?

- ☐ Write Fixed Window Size K in 3m from memory
- ☐ Code LC 3 (Longest Substring)
- ☐ Use `int[128]` for $O(1)$ ASCII freq lookup
- ☐ Code Character Replacement (LC 424)
- ☐ Code Minimum Window Substring (LC 76)
- ☐ Write `atMost(K) - atMost(K-1)` trick
- ☐ Implement Monotonic Deque (LC 239)
- ☐ Prove why Sliding Window is $O(N)$

📐 MATHEMATICAL PROOF — $O(N)$ SLIDING WINDOW

Claim: The two-pointer inner `while` loop does not make the overall time complexity $O(N^2)$.

Proof: The `right` pointer increments N times. The `left` pointer ALSO increments at most N times across the ENTIRE algorithm execution. Total pointer operations $= N + N = 2N \rightarrow$ **Strictly $O(N)$ Time!**

📦 GOLDEN RULES — NEVER FORGET

- Rule 1 — Expand Right, Shrink Left**

Always expand `right` to include next element first. Shrink `left` ONLY when window condition breaks (longest) or while window remains valid (shortest).
- Rule 2 — Monotonic Deque Stores Indices**

In Sliding Window Maximum, store ARRAY INDICES in the Deque, NOT values! Indices allow easy window boundary checks (`q.peek() < i - k + 1`).